Quickstarts

Access USDC crosschain

Copy page

Create a unified Gateway balance on Arc Testnet and transfer that balance to Ethereum Sepolia.

This tutorial shows an Arc-first Gateway flow: deposit USDC from an Arc wallet into a unified Gateway balance on Arc Testnet, then transfer that balance to Ethereum Sepolia.

Gateway supports unified USDC balances across multiple EVM chains. This quickstart shows the Arc Testnet to Ethereum Sepolia flow for clarity. The config objects also include other supported EVM chains that you can swap in as needed.

Select the tab that matches your preferred setup.

For Solana-specific Gateway flows, use the dedicated **Circle Gateway quickstarts**.

Circle Wallets Viem

Prerequisites

Before you begin, make sure you have:

1. A **Circle Developer Console** account
2. An API key created in the Console
3. Your **Entity Secret registered**
4. Testnet USDC and native tokens on Arc Testnet
5. Testnet native tokens on Ethereum Sepolia for the destination mint

Step 1. Set up your project

In this step, you create the project, install dependencies, and configure the environment for the Arc-first Gateway flow.

1.1. Create the project and install dependencies

```
mkdir arc-gateway-balance
cd arc-gateway-balance
npm init -y
npm pkg set type=module
npm pkg set scripts.deposit="tsx --env-file=.env deposit.ts"
npm pkg set scripts.transfer="tsx --env-file=.env transfer.ts"
npm pkg set scripts.balances="tsx --env-file=.env balances.ts"

npm install @circle-fin/developer-controlled-wallets
npm install --save-dev tsx typescript @types/node
```

1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

```
npx tsc --init
```

Then, update the `tsconfig.json` file:

```
cat << 'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Set environment variables

Create a `.env` file in the project directory:

.env

```
CIRCLE_API_KEY=YOUR_API_KEY
CIRCLE_ENTITY_SECRET=YOUR_ENTITY_SECRET
DEPOSITOR_ADDRESS=YOUR_ARC_WALLET_ADDRESS
RECIPIENT_ADDRESS=YOUR_ETHEREUM_ADDRESS
```

- `CIRCLE_API_KEY` is your Circle Developer API key.
- `CIRCLE_ENTITY_SECRET` is your registered Entity Secret.
- `DEPOSITOR_ADDRESS` is the Arc wallet address that deposits into Gateway and signs the burn intent.
- `RECIPIENT_ADDRESS` is the Ethereum Sepolia address that receives the minted USDC.

The `npm run` commands in this tutorial load variables from `.env` using Node.js native env-file support.

Prefer editing `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Set up and fund your wallets

In this step, you create matching dev-controlled EVM wallets on Arc Testnet and Ethereum Sepolia and fund them for the Gateway flow. If you already have funded wallets on both chains, skip ahead to the **deposit step**.

2.1. Create wallets

Create one dev-controlled wallet on Arc Testnet and one on Ethereum Sepolia using the same `refId`. Circle assigns the same EVM address to both wallets, which keeps the source and destination flow easier to follow.

If you want to mint on another supported EVM destination later, create a matching dev-controlled wallet on that chain with the same `refId`, then update `DESTINATION_CHAIN` and `RECIPIENT_ADDRESS` to match.

create-wallets.ts

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/develope
                                                        ›
const client = initiateDeveloperControlledWalletsClient({
  apiKey: process.env.CIRCLE_API_KEY!,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET!,
});

async function createWallets() {
  // Create one wallet set, then derive matching EVM wallets on both chains.
  const walletSet = await client.createWalletSet({
    name: "Arc Gateway Wallets",
  });

  /*
   * Supported EVM testnet wallet blockchains:
   * ETH-SEPOLIA | AVAX-FUJI | MATIC-AMOY | ARB-SEPOLIA
   * UNI-SEPOLIA | BASE-SEPOLIA | OP-SEPOLIA | ARC-TESTNET
   */
  const response = await client.createWallets({
    blockchains: ["ARC-TESTNET", "ETH-SEPOLIA"],
    count: 1,
    walletSetId: walletSet.data?.walletSet?.id ?? "",
    accountType: "EOA",
    metadata: [{ refId: "arc-gateway-depositor" }],
  });

  console.log(JSON.stringify(response.data?.wallets, null, 2));
}

void createWallets();
```

> If you're calling the API directly, you'll need to make two requests: one to create the wallet set and one to create the wallet. Be sure to replace the Entity Secret ciphertext and the idempotency key in your request. If you're using the SDKs, this is handled automatically for you.

Run the script:

```
npx tsx --env-file=.env create-wallets.ts
```

After the wallets are created, update both `DEPOSITOR_ADDRESS` and `RECIPIENT_ADDRESS` in `.env`. You can use the same EVM address for both in this quickstart.

This quickstart uses the same EVM address for `DEPOSITOR_ADDRESS` and `RECIPIENT_ADDRESS`. That way, the same dev-controlled wallet can receive the USDC and submit the mint transaction on Ethereum Sepolia.

2.2. Fund your wallets

To follow this flow:

- fund the Arc Testnet wallet with testnet USDC and native tokens
- fund the Ethereum Sepolia wallet with native tokens for the destination mint

Use the Circle Faucet for Arc Testnet USDC and the Ethereum Sepolia faucet for destination gas.

Step 3. Deposit USDC into a Gateway balance on Arc

In this step, you deposit USDC from Arc Testnet into your unified Gateway balance. The script fixes the source chain to Arc Testnet, but the `SUPPORTED_EVM_CHAINS` object shows the broader EVM support you can extend later.

3.1. Create the shared config file

config.ts

```
export const GATEWAY_WALLET_ADDRESS =
  "0x0077777d7EBA4688BDeF3E311b846F25870A19B9";
export const GATEWAY_MINTER_ADDRESS =
  "0x0022222ABE238Cc2C7Bb1f21003F0a260052475B";
export const SOURCE_CHAIN = "arcTestnet" as const;
export const DESTINATION_CHAIN = "sepolia" as const;
```

··· See all 63 lines

3.2. Create the deposit script

The main logic performs two key actions:

- **Approve USDC transfers**: It calls the `approve` method on the USDC contract to allow the Gateway Wallet contract to transfer USDC from your wallet.
- **Deposit USDC into Gateway**: After receiving the approval transaction hash, it calls the `deposit` method on the Gateway Wallet contract.

deposit.ts

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/develope
import {
  GATEWAY_WALLET_ADDRESS,
  SOURCE_CHAIN,
  SUPPORTED_EVM_CHAINS,
  parseUsdc,
} from "./config.js";

const DEPOSIT_AMOUNT_USDC = "2";

async function waitForTxCompletion(
  client: ReturnType<typeof initiateDeveloperControlledWalletsClient>,
  txId: string,
  label: string,
) {
  const terminalStates = new Set([
    "COMPLETE",
    "CONFIRMED",
    "FAILED",
    "DENIED",
    "CANCELLED",
  ]);

  while (true) {
    const { data } = await client.getTransaction({ id: txId });
    const state = data?.transaction?.state;

    if (state && terminalStates.has(state)) {
      console.log(`${label} final state: ${state}`);
      if (state !== "COMPLETE" && state !== "CONFIRMED") {
        throw new Error(`${label} failed with state ${state}`);
      }
      return;
    }

    await new Promise((resolve) => setTimeout(resolve, 3000));
  }
}

async function main() {
  const chain = SUPPORTED_EVM_CHAINS[SOURCE_CHAIN];
```

```
  const client = initiateDeveloperControlledWalletsClient({
    apiKey: process.env.CIRCLE_API_KEY!,
    entitySecret: process.env.CIRCLE_ENTITY_SECRET!,
  });

  const amount = parseUsdc(DEPOSIT_AMOUNT_USDC);
  console.log(`Approving ${DEPOSIT_AMOUNT_USDC} USDC on ${chain.label}...`);

  // Approve Gateway Wallet to spend USDC
  const approveTx = await client.createContractExecutionTransaction({
    walletAddress: process.env.DEPOSITOR_ADDRESS!,
    blockchain: chain.walletChain,
    contractAddress: chain.usdcAddress,
    abiFunctionSignature: "approve(address,uint256)",
    abiParameters: [GATEWAY_WALLET_ADDRESS, amount],
    fee: { type: "level", config: { feeLevel: "MEDIUM" } },
  });

  const approveTxId = approveTx.data?.id;
  if (!approveTxId) throw new Error("Failed to create approve transaction");
  await waitForTxCompletion(client, approveTxId, "USDC approve");
  console.log(`USDC approve complete on ${chain.label}: ${approveTxId}`);

  console.log(
    `Depositing ${DEPOSIT_AMOUNT_USDC} USDC into Gateway on ${chain.label}...`
  );
  // Deposit USDC into Gateway
  const depositTx = await client.createContractExecutionTransaction({
    walletAddress: process.env.DEPOSITOR_ADDRESS!,
    blockchain: chain.walletChain,
    contractAddress: GATEWAY_WALLET_ADDRESS,
    abiFunctionSignature: "deposit(address,uint256)",
    abiParameters: [chain.usdcAddress, amount],
    fee: { type: "level", config: { feeLevel: "MEDIUM" } },
  });

  const depositTxId = depositTx.data?.id;
  if (!depositTxId) throw new Error("Failed to create deposit transaction");
  await waitForTxCompletion(client, depositTxId, "Gateway deposit");
  console.log(`Gateway deposit complete on ${chain.label}: ${depositTxId}`);
}

void main();
```

Run the deposit script to deposit USDC from Arc Testnet into your Gateway balance.

```
npm run deposit
```

Wait for the required number of block confirmations. Once the Arc deposit transaction is final, the funds become available in your Gateway balance. Note that for certain chains, finality may take up to 20 minutes to be reached.

3.3. Check the Gateway balances

Create a balance script that queries your unified Gateway balance across the configured EVM domains.

balances.ts

```
import { SUPPORTED_EVM_CHAINS } from "./config.js";

interface GatewayBalancesResponse {
  balances: Array<{
    domain: number;
    balance: string;
  }>;
}

async function main() {
  const depositor = process.env.DEPOSITOR_ADDRESS!;
  console.log(`Gateway balances for ${depositor}:`);

  // Query every configured Gateway domain to inspect the unified balance.
  const response = await fetch(
    "https://gateway-api-testnet.circle.com/v1/balances",
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        token: "USDC",
        sources: Object.values(SUPPORTED_EVM_CHAINS).map((chain) => ({
          domain: chain.domainId,
          depositor,
        })),
      }),
    },
  );

  const result = (await response.json()) as GatewayBalancesResponse;

  for (const balance of result.balances) {
    const chain =
      Object.values(SUPPORTED_EVM_CHAINS).find(
        (item) => item.domainId === balance.domain,
      )?.label ?? `Domain ${balance.domain}`;
    console.log(`${chain}: ${balance.balance} USDC`);
  }
}
```

```
void main()
```

Run the balance script to check the balances on the Gateway Wallet.

```
npm run balances
```

Step 4. Transfer USDC from the Arc Gateway balance to Ethereum

In this step, you burn from the Arc Gateway balance and mint on Ethereum Sepolia. The same script shape works for other supported EVM destinations after you create and fund a matching dev-controlled wallet on that chain and update `DESTINATION_CHAIN` and `RECIPIENT_ADDRESS`.

In this flow, `RECIPIENT_ADDRESS` receives the USDC, and `DEPOSITOR_ADDRESS` submits the mint transaction on Ethereum Sepolia. This quickstart keeps them the same.

4.1. Create the transfer script

This example submits one burn intent for clarity. You can also submit multiple burn intents to the Gateway API in a single request and receive a single attestation response to use when minting on the destination chain.

transfer.ts

```
import { randomBytes } from "node:crypto";
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/develope
import {
  DESTINATION_CHAIN,
  GATEWAY_MINTER_ADDRESS,
  GATEWAY_WALLET_ADDRESS,
  SOURCE_CHAIN,
```

··· See all 197 lines

4.2. Run the transfer script

Run the transfer script to transfer 1 USDC from your Arc Gateway balance to Ethereum Sepolia.

```
npm run transfer
```

Summary

After completing this tutorial, you have:

- created or prepared the wallets needed for the Arc-to-Ethereum flow
- deposited USDC from Arc Testnet into a unified Gateway balance
- checked the Gateway balance across supported EVM domains
- transferred that Arc Gateway balance to Ethereum Sepolia

Was this page helpful? Yes No

Transfer USDC or EURC Interact with contracts

Links
Network status
Blog
Community
Arc homepage

Resources
Terms of service
Privacy policy
Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.